CIS 220 Database - Concepts Individual Assignment
Reflexive Relationships and Aggregation

Create a document to contain your work for this assignment. You will submit the document as a PDF when you have finished

TASK ONE: Using the **employees** table from the **Classic Models** database write each employee and the employee they report to. Who is the CEO of the company? How do you know that?

Review the reflexive relationship in **ClassicModels.employees** where the **reportsTo** field is an internal foreign key to the **employeeNumber** field. Any values inserted into or added as an update **reportsTo** must **already exist** in the **employeeNumber**



employeeNumbe	lastName	firstName	extensior	email	officeCode	reportsTo
1002	Murphy	Diane	x5800	dmurphy@classicmodelcars.com	1	NULL
1056	Patterson	Mary	x4611	mpatterso@classicmodelcars.com	1	1002
1076	Firrelli	Jeff	x9273	jfirrelli@classicmodelcars.com	1	1002
1088	Patterson	William	x4871	wpatterson@classicmodelcars.com	6	1056
1102	Bondur	Gerard	x5408	gbondur@classicmodelcars.com	4	1056
1143	Bow	Anthony	x5428	abow@classicmodelcars.com	1	1056
1165	Jennings	Leslie	x3291	ljennings@classicmodelcars.com	1	1143
1166	Thompson	Leslie	x4065	lthompson@classicmodelcars.com	1	1143
1188	Firrelli	Julie	x2173	jfirrelli@classicmodelcars.com	2	1143
1216	Patterson	Steve	x4334	spatterson@classicmodelcars.com	2	1143
1286	Tseng	Foon Yue	x2248	ftseng@classicmodelcars.com	3	1143
1323	Vanauf	George	x4102	gvanauf@classicmodelcars.com	3	1143
1337	Bondur	Loui	x6493	lbondur@classicmodelcars.com	4	1102

TASK TWO – Creature continued: Our team has decided to implement a story line that includes a creature's idolization or "worship" of other creatures in the world. For example, let's say that

- Arthur idolizes The Tick
- Godzilla idolizes Kermit
- Bat Manuel idolizes American Maid
- Thor idolizes no one
- etc etc

The team wants the database to be able to track this special relationship. Design a solution as a 1:M relationship. This relationship can be expressed in the simple language

- A creature may idolize **one other creature**
- A creature can be idolized by **many other creatures**

Using your work from previous assignments, modify your existing **CREATE TABLE** statement for the **creature** table to reflect this design change.

Hint: This is a reflexive association. Review the lesson on Employee/Supervisor and Course/Prerequisites.

Maybe we have creatures that **do not** idolize other creatures. This can easily be implemented by storing NULL in the foreign key.

NOTE: The insertion of the data into this table may be tricky because there will now be a field in the table that refers to another field in the table, and that value **must already exist** if it is to be referred to.

You can have a value of **a reflexive foreign key** that refers to a **primary key** that is inserted at a later time if you use an **UPDATE** statement. You would not be able to INSERT INTO in this order.

Here is an example of updating a field using the Classic Models database

```
USE classicmodels;

UPDATE  employees               -- Table name
SET     reportsTo = 1337        -- field to update
WHERE   employeeNumber = 1102;  -- specify the record to update via PK

-- Changes Gerard Bondur's manager to Loui Bondur
-- Gerard Bondur's employeeNumber is 1102
-- Loui Bondur's employeeNumber is 1337
```

Modify your existing **INSERT INTO** statements for the **Creature** table to reflect this new design. Remember that you may have to UPDATE the table to add the values for the **worships** field **after** you do the inserts.

QUERIES: Once you have completed the structural change to the **Creature** table complete the following queries

SUBMISSION: Screenshot the table descriptions for the updated **creature** table and the results of a **SELECT *** query and the "idol" queries described below. Also screen shot your SQL syntax for the idol query. Place these screenshots into your submission document.

1. Write a query that shows creature name and the creature name of their idol. Do not show any creatures who **do not** have an idol.
2. Write a query that shows all creatures that do not have an idol

Aggregation in SQL – Concept Overview

Aggregation in SQL refers to the process of combining multiple rows of data into a single summary value. This is typically done using **aggregate functions**, which perform calculations across a set of rows.

Common Aggregate Functions:

Function	Description
`COUNT()`	Counts the number of rows
`SUM()`	Adds up numeric values
`AVG()`	Calculates the average value
`MIN()`	Returns the smallest value
`MAX()`	Returns the largest value

Grouping with `GROUP BY`
Aggregate functions normally calculate over *all* rows returned by a query.
To calculate separate summaries for *categories or groups*, SQL uses the **`GROUP BY` clause**.

Example: Count how many students are in each major

```sql
SELECT major_id, COUNT(*)
FROM Students
GROUP BY major_id;
```

Filtering Aggregated Results with `HAVING`

Once data is grouped, `HAVING` can be used to filter groups (similar to how `WHERE` filters individual rows).

Example: Show only majors that have more than 5 students

```sql
SELECT major_id, COUNT(*) AS num_students
FROM Students
GROUP BY major_id
HAVING COUNT(*) > 5;
```

You *can't* use `WHERE` to filter aggregated values like `COUNT(*)` or `SUM()` because `WHERE` is applied *before* aggregation happens.

SQL processes a query in this logical order:

1. **FROM:** get the raw rows
2. **WHERE:** filter *individual rows*
3. **GROUP BY:** group the remaining rows
4. **AGGREGATE:** calculate functions like `SUM`, `COUNT`, `AVG`
5. **HAVING:** filter *groups* based on aggregate results
6. **SELECT:** return the final values

Creature aggregation queries: screen shot your SQL syntax for each query. Place these screenshots into your submission document.

1. Show each creature and a count of the weapons they possess in decreasing order.
2. Show the creature that has the most worshipers.
3. Show the number of creatures that are not worshiped.
4. Show the most popular weapon.
5. Show the town that has the most villains.